

Models for the Two-Dimensional Two-Stage Cutting Stock Problem with Multiple Stock Size

F. Furini, E. Malaguti

Abstract

We consider a Two-Dimensional Cutting Stock Problem (2DCSP) where stock of different sizes is available, and a set of rectangular items has to be obtained through two-stage guillotine cuts. We propose and computationally compare three Mixed-Integer Programming models for the 2DCSP developing formulations from the literature. The first two models have a polynomial and pseudo-polynomial number of variables, respectively, and can be solved with a general-purpose MIP solver. The third model, having an exponential number of variables, is solved via branch-and-price techniques. We conclude the paper describing the results of extensive computational experiments on a set of benchmark instances from the literature.

Keywords: Cutting Stock Problem ; Mixed-Integer Programming Models ; Computational Experiments

1 Introduction

Given a set of small rectangular items and infinitely many larger stock rectangles, the *Two-Dimensional Cutting Stock Problem* requires cutting all the items by minimizing the global area of the used stock. We consider the *Two-Dimensional Two-Stage Cutting Stock Problem with Multiple Stock Size* (denoted as 2DCSP for short) with the following features:

- Items are obtained from the stock rectangle (also denoted as panel) through guillotine cuts, i.e., cuts that are parallel to the sides of the stock and cross the stock from one side to the other;
- Cuts are restricted to two stages, where each stage consists of a set of parallel guillotine cuts performed on the shape obtained in the previous stage; a third stage can be considered for removing a waste area (this cut is known as trimming);
- Stock rectangles (panels) of various sizes are available;
- Without loss of generality, we consider the case in which the first stage cut is horizontal.

On the left part of Figure 1 we give an example of two-stage guillotine cuts on a rectangular bin: first stage cuts (horizontal) are represented with a thick line, second stage cuts (vertical) with a thin line, and trimming cuts (horizontal) with a dotted line. Waste area is represented by a dashed surface. On the right part of the figure we represent a cutting which cannot be obtained through guillotine cuts.

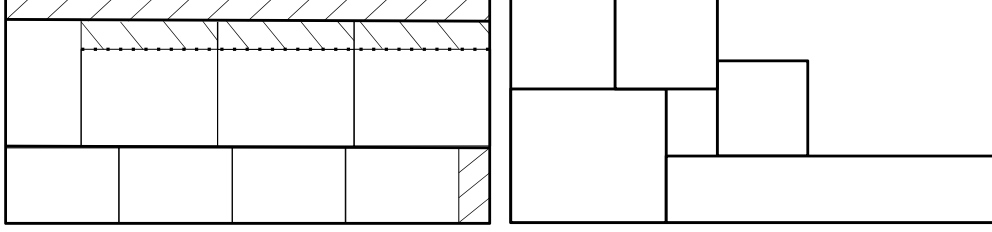


Figure 1: Guillotine cut (left) and non-guillotine cut (right).

Formally, we are given a set of p stock rectangles types, each type h ($h = 1, \dots, p$) with length L_h and width W_h , is available in an infinite number of copies. We are also given a list of n rectangular shapes to be cut. Each shape's type i ($i = 1, \dots, n$) is characterized by a length l_i , a width w_i and a demand d_i . All problem data are integer numbers. The problem is to cut all the items by minimizing the area of the stock rectangles used. According to the typology introduced by Wäscher et al. [Wäscher et al. \(2007\)](#), the problem is the constrained version of the *Two-Dimensional Rectangular Multiple Stock Size Cutting Stock Problem*. The problem is NP-hard and has great relevance in industry whenever a sheet of paper, wood, glass, metal or other material has to be cut.

1.1 Related Literature

The 2DCSP was introduced by Gilmore and Gomory [Gilmore and Gomory \(1965\)](#), who also proposed the k – *stage* version of the problem and the case of stock of different sizes. The authors introduced the well-known exponential-sized model which has to be solved via column generation. This model is at the base of several exact and heuristic approaches (and is reviewed in [Section 2.3](#)).

The literature on heuristic approaches to 2DCSP is very rich, because of the relevance of the problem in industry paired with its computational difficulty, which makes real-word instances very hard to solve to optimality; thus, we restrict our attention to papers dealing with the case of stock of different sizes. Riehme et al. [Riehme et al. \(1996\)](#) considered the two-stage version of the problem in the case where the demands of the item classes differ greatly. The problem is solved in two phases, first taking into account items with large demand, and then solving the residual problem. Alvarez-Valdes et al. [Alvarez-Valdes et al. \(2002\)](#) developed and compared several heuristic methods, based on column generation and rounding of the linear programming relaxation. Cintra et al. [Cintra et al. \(2008\)](#) proposed algorithms for several 2DCSP including the one we consider. Their algorithm generates columns to solve to proven optimality the linear programming relaxation of the Gilmore and Gomory model, which is then iteratively rounded-down so as to obtain an integer feasible solution. Furini et al. [Furini et al. \(2012\)](#) proposed an approach based on column generation and a diving heuristic, improving on the computational results obtained in [Cintra et al. \(2008\)](#). Recently, Malaguti et al. [Malaguti et al. \(2012\)](#) studied a real-world extension of the problem where the objective is to minimize a weighted combination of material usage and number of different patterns in the solution. They proposed heuristic procedures and a truncated branch-and-price algorithm.

As far as exact approaches are concerned, we could not find any papers in the literature explicitly considering the 2DCSP with multiple stock size, although some authors considered close problems, like the Two-Dimensional Bin Packing Problem (2DBPP), that is, a cutting stock problem where the demand of all items is equal to 1. Pisinger and Sigurd [Pisinger and Sigurd \(2005\)](#) considered the 2DBPP with multiple stock size and proposed a branch-and-price algorithm based on the set covering formulation of the problem. Although the 2DCSP is an extension of the 2DBPP, there are some substantial differences in designing a branch-and-price algorithm, namely in the branching strategy to obtain integer solutions and in the column generation subproblem. In a subsequent paper [Pisinger and Sigurd \(2007\)](#), the authors restrict their attention to the 2DCSP with single stock size, and solve the column generation subproblem as a constraint satisfaction problem. Puchinger and Raidl [Puchinger and Raidl \(2007\)](#) propose compact models and a branch-and-price algorithm for the three stage 2DBPP. Finally, a more application-oriented study is presented in Vanderbeck [Vanderbeck \(2001\)](#), where a real-world 2DCSP with multiple stock size and additional features is solved via column generation in an approximate fashion. In the conclusions of the paper, Vanderbeck briefly discusses a possible extension of the approach to an exact branch-and-bound method, but no details or computational results are given.

In terms of optimization models not based on the Gilmore and Gomory formulation, we mention the model proposed for the two-stage 2DBPP by Lodi et al. [Lodi et al. \(2004\)](#), which has a polynomial number of variables and constraints. We extend this model to the CS case with multiple stock size in Section 2.1. Macedo et al. [Macedo et al. \(2010\)](#) solved the two-stage 2DCSP by extending an Integer Linear Programming formulation proposed by Valério de Carvalho [de Carvalho \(2002\)](#) for the one dimensional CSP. The extension of the model to two-dimensions requires defining a set of flow problems to determine a set of horizontal strips, and a flow problem to determine how the strips fit into the stock rectangle. Finally, Silva et al [Silva et al. \(2010\)](#) present a pseudo-polynomial size model for the two and three-stage 2DCSP based on the notion of cuts and residual plates, obtained after the cut. This model and its extension to the case with multiple stock size are presented in Section 2.2. For a more comprehensive survey on 2D cutting and packing the reader is referred to Lodi et al. [Lodi et al. \(2002\)](#) and Wäscher et al. [Wäscher et al. \(2007\)](#).

1.2 Paper Contributions

In this paper we computationally compare three Mixed-Integer Programming models (MIPs) for the 2DCSP. The first model (Section 2.1) is an extension, to the Cutting Stock case with multiple stock size, of the Bin Packing model by Lodi et al. [Lodi et al. \(2004\)](#). In Section 2.2 we describe the second model, proposed by Silva et al. [Silva et al. \(2010\)](#). The first two models have a polynomial and pseudo-polynomial number of variables, respectively, which can be exhaustively enumerated before solving the problem with a general-purpose MIP solver. Finally, in Section 2.3 we give the third model, which is the adaptation to the case with multiple stock size of the exponential size model proposed by Gilmore and Gomory, and needs branch-and-price techniques to be solved. In Section 3 we compare the three models through extensive computational experiments on a set of benchmark instances from the literature.

2 Cutting Stock Models

This Section describes the three MIP models that are the object of our computational study.

2.1 Compact formulation

Lodi, Martello and Vigo [Lodi et al. \(2004\)](#) proposed a compact model for the Two-Stage Two-Dimensional Bin Packing Problem. The model is based on the observation that two-stage cutting produces shelves (or levels) during the first stage cutting, i.e., slices of the stock rectangle with the same length, and width coincident with the width of the widest item cut off from it. In the model, items are sorted so that $w_1 \geq w_2 \geq \dots \geq w_n$. The maximum number of levels that would result from the first phase cutting is n ; in the model, each level is associated with a different item i which initializes it. This means that a level i can be selected in a solution only if it contains item i (item i is the widest item in the level, and determines the level width). Similarly, the maximum number of stock panels that would be used is also n . Each stock panel is associated with the level of same index, meaning that a panel i can be selected in a solution only if it contains level i . Model [Lodi et al. \(2004\)](#) can be used for solving cutting stock problems by replacing each item class i with d_i identical items. However, the duplication of identical items would introduce much symmetry and would make the model extremely difficult to solve by means of a general purpose MIP solver, because of its several equivalent solutions.

Instead of duplicating items, we improve the model by considering the items with the same shape's type together, but separating them with respect to the initialization of the shelves, as done by Lodi and Monaci for solving the two-stage 2D Knapsack Problem (2DKP) [Lodi and Monaci \(2003\)](#). In addition, we extend the model to the case where the stock is available in panels of various sizes, denoted as panel classes. In the model we minimize the total area of the used stock (in the following we denote by A_h the area of a panel of class h , and we use the parameters defined in Section 1 for the formal statement of the problem).

We denote by $\tilde{n} = \sum_{j=1,\dots,n} d_j$ the total number of items. By keeping the assumption that items are sorted according to non-increasing width, and by considering a specific panel class, any item of type j may be cut from shelves in $[1, \alpha_j]$, with $\alpha_j = \sum_{s=1}^j d_s$ ($j = 1, \dots, n$) and $\alpha_0 = 0$. Conversely, any shelf k associated with a given panel class can be used to cut items in classes $[\beta_k, n]$, with $\beta_k = \min\{r : 1 \leq r \leq n, \alpha_r \geq k\}$ ($k = 1, \dots, \tilde{n}$). Thus, β_k denotes the item type initializing shelf k . The model, denoted as Compact formulation in the following, uses four sets of variables. Binary variables, y_i^h , which take value 1 if item i initializes level i in a panel of class h ($i = 1, \dots, \tilde{n}; h = 1, \dots, p$). Integer variables, x_{ij}^h , which represent the number of (additional) items of type j packed into level i in a panel of class h ($i = 1, \dots, \tilde{n} - 1; j = \beta_i, \dots, n; h = 1, \dots, p$). Binary variables, q_k^h , which take value 1 if level k initializes panel k of class h ($k = 1, \dots, \tilde{n}; h = 1, \dots, p$). Finally binary variables, z_{ki}^h , which take value 1 if level i is allocated to panel k of class h ($k = 1, \dots, \tilde{n} - 1; i = k, \dots, \tilde{n}; h = 1, \dots, p$). The Compact formulation reads as follows:

$$\min \sum_{h=1}^p A^h \sum_{k=1}^{\tilde{n}} q_k^h \quad (1)$$

$$\sum_{h=1}^p \left(\sum_{i=1}^{\alpha_j} x_{ij}^h + \sum_{i=\alpha_{j-1}+1}^{\alpha_j} y_i^h \right) \geq d_j \quad j=1, \dots, n \quad (2)$$

$$\sum_{j=\beta_i}^n l_j x_{ij}^h \leq (L_h - l_{\beta_i}) y_i^h \quad i=1, \dots, \tilde{n}-1; h=1, \dots, p \quad (3)$$

$$\sum_{k=1}^{i-1} z_{ki}^h + q_i^h = y_i^h \quad i=1, \dots, \tilde{n}; h=1, \dots, p \quad (4)$$

$$\sum_{i=k+1}^{\tilde{n}} w_{\beta_i} z_{ki} \leq (W_h - w_{\beta_k}) q_k^h \quad k=1, \dots, \tilde{n}-1; h=1, \dots, p \quad (5)$$

$$y_i^h \in \{0, 1\} \quad i=1, \dots, \tilde{n}; h=1, \dots, p \quad (6)$$

$$x_{ij}^h \in \mathbb{Z}_+ \quad i=1, \dots, \tilde{n}-1; j=\beta_i, \dots, n; h=1, \dots, p \quad (7)$$

$$q_k^h \in \{0, 1\} \quad k=1, \dots, \tilde{n}; h=1, \dots, p \quad (8)$$

$$z_{ki}^h \in \{0, 1\} \quad k=1, \dots, \tilde{n}-1; i=k, \dots, \tilde{n}; h=1, \dots, p \quad (9)$$

The objective function (1) minimizes the total area of the used stock panels. Constraints (2) impose that the demand of each item class is satisfied, either by including the items in some level where they can fit, or by initializing new levels. Constraints (3) impose the satisfaction of the horizontal capacity for a level i of a panel of type h , when initialized by the corresponding item β_i . Constraints (4) force each initialized level i of a stock panel of class h to either initialize a panel having the same index, or to be included in another panel (of the same type) of smaller index. Finally, Constraints (5) impose the satisfaction of the vertical capacity of a stock rectangle k of type h , when initialized by the corresponding level k . Constraints (6)–(9) impose all variables to be binary, except the x variables that are integer.

With respect to the model presented in Lodi et al. (2004), we defined the index h to denote the class of the panel. In addition, since in Lodi et al. (2004) each item appears in a single copy, all the variables are binary. Thus, the Compact formulation simplifies to the model presented in Lodi et al. (2004) by considering a single class of panels and by defining *binary* variables x_{ij} taking value 1 if item j is packed into level i . Any item of type j may be cut from shelves in $[1, j]$, (i.e., $\alpha_j = j$), and any shelf k can be used to cut items in classes $[k, n]$ (i.e., $\beta_k = j$).

The LP relaxation of the Compact formulation is allowed to split an item into one part initializing the shelf (y part) and some others which can be packed as “additional” parts (x parts). For example, we may have that item 1 partially initializes the first level ($y_{11} = 1/2$) and partially is included in the first level as an additional item ($x_{11} = 1/2$) (we drop index h without loss of generality). The item is then possibly taken into account completely, while the width of the level is only partially paid. In the example, the first item appears in the first strip with value 1, but the total width of the strip, which consumes the panel capacity through equations (4) and

(5) (where the x variables do not appear), is only $1/2w_1$. We add a set of inequalities (proposed in [Lodi and Monaci \(2003\)](#) for the 2DKP) to avoid this situation:

$$\sum_{s=k}^{\alpha_j} x_{sj}^h \leq d_j - (k - \alpha_{j-1}) \quad (j = 1, \dots, n; \quad k \in [\alpha_{j-1} + 1, \alpha_j]; \quad h = 1, \dots, p)$$

2.2 Pseudo-polynomial formulation

Dyckhoff [Dyckhoff \(1981\)](#) proposed a compact model for the Cutting Stock problem, based on the concept of cut. The idea is the following: each time a one-dimensional stock of length L is cut in order to produce an item i of length l_i , a residual element of size $L - l_i$ is obtained, which can be further used to produce additional items. The model associates a decision variable to each item and each possible (original or residual) element. Feasible solutions are obtained by imposing balance constraints on the number of residual elements, while the cost of a solution is obtained by counting the number of used original elements (stock).

The idea was extended by Silva, Alvelos and Valério de Carvalho [Silva et al. \(2010\)](#) to two and three-stage two-dimensional Cutting Stock Problems. In the two-dimensional case we consider pairs of items and plates, where a plate can be the original panel or a sub-panel obtained as the result of a previous cut. Each time an item i is obtained through two guillotine cuts (the first one horizontal, the second one vertical) from a plate j of length L_j and width W_j where it fits, we obtain two new plates, the first one having length $L_j - l_i$ and width w_i , and the second one having length L_j and width $W_j - w_i$. The two obtained plates can be used in a subsequent cut or can be waste depending on their size and on the number of stages of the cutting procedure. The model, denoted as Pseudo-polynomial formulation in the following, has decision variables x_{ij} defined as the number of times an item of type i is cut from a plate of type j . The first p plate types correspond to the p original stock panels, whose used area we want to minimize. Parameter a_{ijk} is equal to 1 if plate type k results from cutting item type i from plate type j , and equal to 0 otherwise. The model is initialized by enumerating all feasible item-plate pairs. Let m be the total number of plates (including the original p stock panels). The Pseudo-polynomial formulation reads:

$$\min \sum_{h=1}^p A^h \sum_{i=1}^n x_{ih} \tag{10}$$

$$\sum_{j=1}^m x_{ij} \geq d_i, \quad i = 1, \dots, n \tag{11}$$

$$\sum_{j=1}^m \sum_{i=1}^n a_{ijk} x_{ij} \geq \sum_{i=1}^n x_{ik} \quad k = 1, \dots, m \tag{12}$$

$$x_{ij} \in \mathbb{Z}_+ \quad i = 1, \dots, n, \quad j = 1, \dots, m \tag{13}$$

Where (11) impose that the demand of each item is satisfied, and (12) impose that the number

of used plates of type k is not larger than the number of plates of the same type obtained by previous cuts. Model (10)–(13) extends the formulation presented in Silva et al. (2010) to the case of various stock size by considering in the objective function (10) all stock sizes originally available, each one weighted according to its area. The number of cuts and, consequently, the number of variables and constraints of the model, is pseudo-polynomial, depending on W and H . We refer the reader to Silva et al. (2010) and the results therein for further details.

2.3 Exponential formulation

The exponential-size model for the CSP was introduced by Gilmore and Gomory (Gilmore and Gomory (1965)), but we are not aware of any implementation or detailed description of a branch-and-price algorithm for solving the 2DCSP with multiple stock size. Let S^h ($h = 1, \dots, p$) be the family of all the feasible cutting patterns for stock class h , that is, cutting patterns which do not exceed the size of stock panel h and can be obtained by two-stage guillotine cuts. In addition, we consider only patterns which do not contain more items of an item class i than specified by the corresponding request d_i . The 2DCSP can be modeled by using integer variables x_s associated with each feasible cutting pattern $s \in S^h$. The value of each of these variables denotes the number of times the corresponding pattern is used in the solution. The objective function minimizes the total area of the used stock rectangles. By introducing a parameter a_s^i to represent the number of items of class i in the cutting pattern s , the Exponential formulation of the 2DCSP reads:

$$\min \sum_{h=1, \dots, p} A^h \sum_{s \in S^h} x_s \quad (14)$$

$$\sum_{h=1, \dots, p} \sum_{s \in S^h} a_s^i x_s \geq d_i \quad i = 1, \dots, n \quad (15)$$

$$x_s \in \mathbb{Z}^+ \quad s \in S^h, \quad h = 1, \dots, p \quad (16)$$

By dropping the integrality requirement for the variables, imposed by constraints (16), we obtain the Linear Programming (LP) relaxation of the model, defined by (14), (15) and:

$$x_s \geq 0, \quad s \in S^h, \quad h = 1, \dots, p, \quad (17)$$

and denoted as *Master Problem* (MP). We initialize a *restricted* MP with a subset of the x_s variables; new variables, needed to solve MP to optimality, are then obtained by separating the following dual constraints:

$$\sum_{i=1}^n a_s^i \pi_i^* \leq A^h \quad s \in S^h, \quad h = 1, \dots, p, \quad (18)$$

where π_i^* ($i = 1, \dots, n$) represents the optimal dual variable associated with the i -th constraint

of type (15).

Accordingly, the so-called *Slave Problem* (SP) is to determine a stock class h and a feasible pattern $s^* \in S^h$, if one exists, having negative reduced cost (i.e. violating the dual constraints (18)):

$$\sum_{i=1}^n a_{s^*}^i \pi_i^* > A^h \quad (19)$$

A slave problem is solved for each stock class h ; the slave problem can be modeled as a Bounded two-dimensional two-stage Knapsack Problem (B2DKP), where the profit associated with item i equals the value of the corresponding dual variable π_i^* . When new variables with negative reduced cost are generated, they are added to the MP, which is then re-optimized, according to a classic column generation scheme. If no column with negative reduced cost exists, the MP is optimally solved. In some cases the solution of the MP is integer, and thus optimal also for the Exponential formulation. Otherwise, the solution value of the MP is a lower bound for the problem; since all problem data are integer numbers, this value is rounded-up to improve the bound. When considering the case with various stock size, the lower bound can be further improved: we observe that the objective function for integer solutions can only take values that are combinations of the areas of the panels. Thus, after rounding we solve a Subset-Sum problem (see, e.g., Martello and Toth [Martello and Toth \(1990\)](#)) in order to find the closest integer value that is a combination of the areas of the panels and is larger than the rounded-up solution value of the MP. Instead, when all panels are identical, before solving the model we scale the objective function and the cost of each panel is set to 1. This is equivalent to minimizing the total number of used panels.

Finally, an integer optimal solution for the problem can be found by embedding the column generation within a branching scheme, thus obtaining a branch-and-price algorithm. Note that the Subset-Sum problem is then solved at each node of the branch-and-price algorithm, in order to strengthen the associated lower bound.

2.3.1 Branching

When branching in the one dimensional CSP, a possibility would be to map the items i with demand $d_i > 1$ to $\lfloor \log d_i \rfloor + 1$ separated items i_l , $l = 0, \dots, \lfloor \log d_i \rfloor$, thus having *integer* columns of original items a_s in the MP and *binary* columns of “new” items $\bar{a}_s$ in the slave: $a_s^i = \sum_{l=0}^{\lfloor \log d_i \rfloor} 2^l a_{\bar{s}}^l$. We denote the correspondence between a column s in the MP with a column of “new” items $\bar{s}$ with the notation $s \leftrightarrow \bar{s}$. Then, one can branch by selecting a “new” item i_l such that $\sum_{s \leftrightarrow \bar{s}: i_l=1} x_s = \alpha$ is fractional, and by imposing, through an additional constraint in the MP, the integrality of α . After branching, in the slave problem one would have to modify the dual variable π_{i_l} associated with item i_l (see, e.g., Vanderbeck [Vanderbeck \(1999, 2000\)](#)).

In the one dimensional case it is enough to collapse 2, 4, 8, etc. copies of the original items in

order to define the “new” items, while in two dimensions it is not clear how the original items should be merged, and thus there is not a straightforward extension of this idea.

Considering now the 2DBPP, it is possible to select two items i and j ($i \neq j$), and to branch by considering on the one side only columns where i and j appear together, and on the other side only columns that do not contain both i and j . This strategy makes a subset of the MP columns infeasible after branching, and additional constraints are added to the slave problems in order not to re-generate “forbidden” columns. Again, there is not a straightforward extension of this idea to the CS case, where items appear with demand larger than 1.

A way of branching for the 2DCSP, suggested in Vanderbeck [Vanderbeck \(1999, 2000\)](#), is the following: find an item i , a value $v \in \{1, \dots, d_i\}$ and a “sense” $\geq$ or $<$ such that $\sum_{s:a_s^i \geq v} x_s = \alpha$ is fractional (or, eq., consider $\sum_{s:a_s^i < v} x_s$). Then branch by imposing, in the MP, the following *branching constraints*:

$$\sum_{s:a_s^i \geq v} x_s \leq \lfloor \alpha \rfloor \quad (20)$$

$$\sum_{s:a_s^i < v} x_s \geq \lceil \alpha \rceil \quad (21)$$

Note that this branching rule is applied to the case of multiple stock size without distinguishing between stock panel types. In general we may need more than a triple item/level/sense, called *component bound*, to find a fractional α . Let $f = \sum_{s \in S} x_s - \lfloor x_s \rfloor$ and $\mathcal{S}(B)$ a set of columns satisfying a set of component bounds B . The number of component bounds needed for branching is bounded by the following result:

Proposition 2.1. (Vanderbeck [Vanderbeck \(2000\)](#)). Given a fractional solution x , there exists a set of component bounds B with $|B| \leq \lceil \log f \rceil + 1$ such that $\sum_{s \in \mathcal{S}(B)} x_s$ is fractional.

In practice, the situation is even better: in the majority of the cases, a one component bound suffices to find a fractional α value for branching, and a two component bound is always enough (see Section 3).

When solving the problem for a single type of stock panel, the cost of each panel is scaled to 1 and the objective function (14) simply counts the number of used panels. In this case, we add the following valid inequality to the MP and to each node of the branch-and-price:

$$\sum_{s \in S} x_s \geq LB_{node} \quad (22)$$

where LB_{node} is the lower bound at the node, i.e., the rounded-up value of the optimal solution of the MP at the node. We denote by ψ the associated dual variable. Assuming that the LB_{node} bound is tight, we can thus omit component bounds B of kind $<$ because they are implied as complements of $\geq$.

2.3.2 Slave Problem Modification

Let $K = K1 \cup K2$ be the set of branching constraints (20) and (21), and μ_k , $k \in K1$ and ν_k , $k \in K2$ the associated dual variables. Let $i(k)$, $v(k)$ be the item and value associated with branching constraint k . In the following we only consider component bounds of kind $\geq$. At a generic node of the branch-and-price, the slave problem can be modeled by using the following variables: z_j , $j = 1, \dots, n$, denoting the number of times the item j is included in the new pattern; g_k , $k \in K^1$ are binary variables used to decide whether the pattern falls into the condition specified by the corresponding branching constraint k , and thus it is assigned the associated penalty μ_k ; h_k , $k \in K^2$ are binary variables used to decide whether the pattern falls into the condition specified by the corresponding branching constraint k , and thus is assigned the associated award ν_k . Given an optimal dual solution $(\pi^*, \mu^*, \nu^*, \psi^*)$ of the MP, the slave problem can be modeled as follows:

$$\max \sum_{j=1}^n \pi_j^* z_j - \sum_{k \in K^1} g_k \mu_k^* + \sum_{k \in K^2} h_k \nu_k^* \quad (23)$$

$$(d_{i(k)} - v(k) + 1)g_k \geq (z_{i(k)} - v(k) + 1) \quad k \in K^1 \quad (24)$$

$$v(k)h_k \leq z_{i(k)} \quad k \in K^2 \quad (25)$$

$$[\text{B2DKP constraints}] \quad (26)$$

$$0 \leq z_j \leq d_j \text{ integer} \quad j = 1, \dots, n \quad (27)$$

$$g_k \in \{0, 1\} \quad k \in K^1 \quad (28)$$

$$h_k \in \{0, 1\} \quad k \in K^2 \quad (29)$$

The objective function (23) maximizes the profit of the selected items, which is amended of the penalty or award corresponding to the dual variables associated with the branching constraints. When the profit of the cutting pattern is larger than A^h , or $1 - \psi^*$ when considering a single type of panel, we have a negative reduced cost column to be added to the MP. Constraints (24) impose that the objective function is penalized by a value μ_k^* when the cutting pattern contains $v(k)$ or more copies of item $i(k)$; conversely, constraints (25) let the objective function earn the additional profit ν_k^* only if the cutting pattern contains $v(k)$ or more copies of item $i(k)$. Finally, we need to impose that the cutting pattern represents a feasible filling for the panel, which is obtained through constraints (26). Similarly (but at cost of additional variables and constraints) we can tackle the case where more than one component bounds are needed for branching. We refer the reader to Vanderbeck [Vanderbeck \(1999, 2000\)](#) for further details. Note that this branching may not remove fractional solutions in the case of variable stock size, when different panels are filled with the same items (such a solution can be optimal for the MP only if the panels with identical filling have the same cost).

2.4 Implementation details

The master problem is initialized with the columns corresponding to the best solution found by the heuristic algorithm described in [Furini et al. \(2012\)](#) with a time limit of 10 seconds. In order to have balanced branching trees, given an item i , when choosing a value v for a component bound, it is desirable for it to be far from its extreme values, namely 1 and $\min\{d_i, \max_{h=1,\dots,p} UB(i, h)\}$, where $UB(i, h)$ is an upper bound on the number of items of type i that can fit in a stock panel of type h . Given a fractional solution to the MP, when searching for a component bound such that $\sum_{s: a_s^i \geq v} x_s = \alpha$ is fractional, for each item i we enumerate all possible values of v , by starting from $v_{init}(i) = \lfloor \frac{\min\{d_i, \max_{h=1,\dots,p} UB(i, h)\}}{2} \rfloor$ and in sequence considering $v_{init}(i) + 1$, $v_{init}(i) - 1$, etc. When a one component bound is not enough for branching, we use a two component bound, that is, we consider two items i and t and two associated levels v and w . The Branch-and-Bound tree that we obtain is explored according to a depth-first strategy.

For solving the slave problem, we use the compact model M2, proposed by Lodi and Monaci [Lodi and Monaci \(2003\)](#) for the 2DKP. The model is modified as previously discussed, and solved with a general purpose MIP solver. We implemented a callback function, and the optimization of the slave is stopped as soon as a column having negative reduced cost is found. In order to reduce the computational effort for solving the slave problems, the instances were preprocessed and new demands for stock panel h defined as $\tilde{d}_j = \min\{d_j, UB(j, h)\}$, $j = 1, \dots, n$.

3 Computational experiments

We performed an extensive computational evaluation of the performances of the different mathematical formulations presented in the previous sections, i.e. the Compact formulation of section 2.1, the Pseudo-polynomial formulation of section 2.2 and the Exponential formulation of section 2.3. All MIP models and linear programs were solved by using Cplex 12.4 [v12.4](#) with default parameters and in single thread mode. The experiments were run on a PC with an Intel i5-650 at 3.20GHz and 8 GB of RAM memory, under Linux openSUSE (64-bit).

3.1 Instances features

We consider two different sets of instances from the literature, testing the case of multiple stock panels (*Multi*). Since all the formulations can easily be adapted to solve the case of a single stock panel (*Mono*), we close the computational section reporting these results as well.

The first set consists of the 12 instances (called $gcut1d, \dots, gcut12d$) proposed by Cintra et al. [Cintra et al. \(2008\)](#) and publicly available online¹. These instances are obtained by adding a random integer demand in the range $[1, \dots, 100]$ to the Two Dimensional Knapsack Problem (2DKP) instances $gcut1, \dots, gcut12$ from the OR library [OR-Library](#). The *Multi* case was defined by Cintra et al. [Cintra et al. \(2008\)](#) by considering three stock panel classes for each instance: the first class is the original one of dimensions (L, W) , the other two classes have dimensions $(1.2L, 0.8W)$ and $(1.1L, 0.9W)$, respectively.

¹<http://www.ime.usp.br/yw/BinPackingData/>

The second set comprises 30 instances of 2DKP proposed in [Hifi and Roucairol \(2001\)](#) and publicly available online². These instances already present a demand for each item. In that paper there are weighted instances and unweighted instances, where the weight is represented by the item area. Since we are minimizing the total area, the item weights are not used here. Following what Cintra et al. [Cintra et al. \(2008\)](#) did for the other class of instances, we generated the *Multi* case with the same rules.

The features of all instances are summarized in Table 1, where we report the number n of item classes, the total number $\tilde{n}$ of items, the original stock panel dimensions L and W , the minimum and maximum item lengths l_{min} and l_{max} and widths w_{min} and w_{max} . Finally the last 2 columns of the table, opt_{Mono} and opt_{Multi} , represent the best solution obtained by the mathematical formulations. For the case *Mono* the value is expressed in terms of number of stock panels while for the *Multi* the value represents the total area. We report in bold the instances solved to proven optimality by at least one of the mathematical formulations presented in this paper. For the *Mono* case all the 42 instances are solved while, for the *Multi* case, 21 instances out of 44 were solved to optimality.

3.2 Comparison between the three mathematical formulations

Table 2 concerns the *Multi* case and we report for each instance, and for the Compact formulation, the Pseudo-polynomial formulation and the Exponential formulation respectively, the number of branch-and-bound (or branch-and-price) nodes and the computing time expressed in seconds (tl in case of time limit of 1800 seconds). Then the table reports three different gaps for each formulation, computed with respect to the following values: opt , which is the best solution value computed by all the formulations (see Table 3.1), lp , which is the continuous relaxation of the specific formulation, $best$, which is the best solution found by the specific formulation and finally bd , which is the bound found by the specific formulation in case of time limit (lp and bd coincide for the Exponential formulation since the bound is not updated in the branch-and-price tree). The formulas used to compute the different gaps are:

- $LP_{gap} = 100 \frac{opt - lp}{opt}$;
- $OPT_{gap} = 100 \frac{best - bd}{best}$;
- $SOL_{gap} = 100 \frac{opt - best}{opt}$.

LP_{gap} is a measure of the quality of each formulation in terms of continuous relaxation; OPT_{gap} is the integrality gap; and SOL_{gap} is a measure of the quality of the feasible solutions produced by each formulation. According to the experiments, there is no clear general winner between the three formulations. Each formulation outperforms the others for some specific instances. All the formulations reached the time limit for many instances. The Compact formulation was able to solve 10 instances, the Pseudo-polynomial formulation solved 20 instances, and the Exponential formulation solved 7 instances. Concerning the computing times, the three

²<ftp://panoramix.univ-paris1.fr/pub/CERMSEM/hifi/>

<i>instance</i>	<i>n</i>	$\tilde{n}$	<i>L</i>	<i>W</i>	l_{min}	l_{max}	w_{min}	w_{max}	<i>opt_{Mono}</i>	<i>opt_{Multi}</i>
<i>gcut1d</i>	10	669	250	250	66	167	86	184	295	14871875
<i>gcut2d</i>	20	982	250	250	18	168	68	186	345	16755000
<i>gcut3d</i>	30	1489	250	250	63	176	71	186	342	20174375
<i>gcut4d</i>	50	2751	250	250	62	184	63	179	845	46529375
<i>gcut5d</i>	10	645	500	500	132	364	145	356	207	41697500
<i>gcut6d</i>	20	1064	500	500	132	355	134	372	375	77637500
<i>gcut7d</i>	30	1626	500	500	129	365	147	374	600	123975000
<i>gcut8d</i>	50	2363	500	500	131	362	127	374	720	161085000
<i>gcut9d</i>	10	590	1000	1000	292	673	341	688	135	131310000
<i>gcut10d</i>	20	830	1000	1000	269	730	260	742	315	260920000
<i>gcut11d</i>	30	1298	1000	1000	266	674	274	745	349	303330000
<i>gcut12d</i>	50	2081	1000	1000	254	746	269	723	675	609780000
<i>HH</i>	5	18	127	98	18	54	13	65	2	23712
<i>CW1</i>	25	67	125	105	25	78	21	66	10	126834
<i>CW2</i>	35	63	145	165	34	93	34	104	12	281529
<i>CW3</i>	40	96	267	207	59	170	45	130	16	857457
<i>Hchl2</i>	35	75	130	130	19	57	18	59	6	98527
<i>Hchl9</i>	35	76	65	76	10	31	10	43	10	51480
<i>2s</i>	10	23	40	70	9	31	7	35	2	5460
<i>3s</i>	20	62	40	70	9	33	11	43	23	63112
<i>A1s</i>	20	62	50	60	9	33	11	43	23	49980
<i>A2s</i>	20	53	60	60	12	33	14	42	12	41832
<i>STS2S</i>	30	78	55	85	10	31	10	43	12	54331
<i>STS4S</i>	20	50	99	99	14	44	16	49	5	47190
<i>OF1</i>	10	23	70	40	9	55	4	39	4	10864
<i>OF2</i>	10	24	70	40	13	47	4	27	5	10836
<i>W</i>	20	62	70	40	11	43	9	33	24	54012
<i>CHL1S</i>	30	63	132	100	13	69	12	63	6	76810
<i>CHL2S</i>	10	19	62	55	11	31	9	31	3	9768
<i>A3</i>	20	46	70	80	15	35	14	43	8	37968
<i>A4</i>	20	35	90	70	9	33	11	43	5	30429
<i>A5</i>	20	45	132	100	13	69	12	63	5	52500
<i>CHL5</i>	10	18	20	20	1	20	2	14	4	1164
<i>CHL6</i>	30	65	130	130	18	69	12	63	6	97344
<i>CHL7</i>	35	75	130	130	19	57	18	54	6	98020
<i>CU1</i>	35	90	150	175	31	96	35	112	15	363315
<i>CU2</i>	25	82	100	125	20	58	28	80	12	145780
<i>Hchl3s</i>	10	51	127	98	15	54	13	65	3	35944
<i>Hchl4s</i>	10	32	127	98	15	54	13	65	2	24088
<i>Hchl6s</i>	22	60	253	244	35	109	38	101	5	297222
<i>Hchl7s</i>	40	90	263	241	33	108	38	135	7	489672
<i>Hchl8s</i>	10	18	49	20	1	20	2	14	2	1856

Table 1: Instance Features

formulations behaved better for different instances. The Compact formulation was faster than the others in three instances, the Pseudo-polynomial formulation was faster in 15 instances, and the Exponential formulation was faster in 3 instances. As for LP_{gap} , the Exponential formulation always outperforms as expected the other two, showing the best linear programming relaxation. The Pseudo-polynomial formulation performed as well as Exponential formulation in only two cases. For OPT_{gap} , the Compact formulation is the best performer in 3 cases, the Pseudo-polynomial formulation in 24 cases, and the Exponential formulation in 6 cases. For SOL_{gap} , the Compact formulation is the best performer in 5 cases, Pseudo-polynomial formulation in 20 cases, while Exponential formulation never wins. In Table 2 we report in bold the entries corresponding to the winning formulation (all the columns except the nodes). To summarize all the results, the Pseudo-polynomial formulation tends to outperform the others as far as the computing time is concerned, but there are some exceptions in which the Compact formulation and the Exponential formulation win. In terms of quality of the linear programming relaxation, the Exponential formulation is the clear winner.

More details about the structure of the mathematical formulations are given in Table 3. We report the number of variables (*vars*) and the number of constraints (*cons*) for the Compact formulation and the Pseudo-polynomial formulation. Finally, for the Exponential formulation we report the total number of generated columns (*cols*) and the number of columns generated in the nodes of the branch-and-price tree (*cols_N*). The last column ($|B| = 2$) represents the percentage of times a two component bound was necessary for branching. From the table it is clear why the Compact formulation behaves poorly for the instances of the first instance set, that is, the large number of items that implies a huge amount of variables and constraints. Actually, this formulation defines a potential strip and a potential panel for each item.

Table 4 concerns the *Mono* case and reports the same values described for Table 2. The Exponential formulation is not included in this table because the other two formulations outperformed it significantly. As far as the first two formulations are concerned, there is no clear winner in terms of computing time, but the Pseudo-polynomial formulation tends to perform better. The Compact formulation needs shorter computing time in 8 of the cases while the Pseudo-polynomial formulation needs shorter time in the remaining 34 cases. It is important to notice that for the second set of instances, both formulations showed almost the same behavior in terms of computing time, although the Pseudo-polynomial formulation reached the time limit of 600 seconds in one case (instance Hchl7s). As for the first set of instances, the Pseudo-polynomial formulation clearly outperforms the Compact formulation, solving to proven optimality all 12 instances while the Pseudo-polynomial formulation reaches the time limit in 7 cases. In terms of LP_{gap} , Pseudo-polynomial formulation is the winning formulation, often showing a small gap that cplex is able to close within a few branching nodes. Finally, it is worth noting that the Pseudo-polynomial formulation is beaten by the Compact formulation in a few instances, despite presenting the smallest average LP_{gap} .

instance	Compact formulation					Pseudo-polynomial formulation					Exponential formulation				
	nodes	time	LP_{gap}	OPT_{gap}	SOL_{gap}	nodes	time	LP_{gap}	OPT_{gap}	SOL_{gap}	nodes	time	LP_{gap}	OPT_{gap}	SOL_{gap}
<i>gcut1d</i>	143825	tl	21.52	0.33	0.00	477	0.1	0.33	0.01	0.00	39972	tl	0.33	0.33	0.00
<i>gcut2d</i>	801	tl	15.82	0.46	0.37	37	0.1	0.08	0.01	0.00	19630	tl	0.08	0.08	0.00
<i>gcut3d</i>	373	tl	8.95	3.58	2.99	1351104	tl	0.12	0.06	0.00	8470	tl	0.12	0.14	0.02
<i>gcut4d</i>	6	tl	8.34	89.72	89.70	515	6.5	0.01	0.01	0.00	2437	tl	0.01	0.10	0.08
<i>gcut5d</i>	2194	tl	14.13	0.16	0.02	0	0.0	0.07	0.00	0.00	44028	tl	0.07	0.09	0.02
<i>gcut6d</i>	20401	tl	16.48	0.01	0.00	0	0.0	0.02	0.01	0.00	17478	tl	0.02	0.24	0.22
<i>gcut7d</i>	513	tl	14.05	0.23	0.20	668	0.7	0.02	0.00	0.00	4246	tl	0.02	0.20	0.18
<i>gcut8d</i>	0	tl	10.57	1.81	1.70	524	4.6	0.01	0.01	0.00	2660	tl	0.01	0.02	0.01
<i>gcut9d</i>	35583	tl	14.11	0.41	0.02	12270087	tl	0.39	0.29	0.00	21872	tl	0.38	0.65	0.27
<i>gcut10d</i>	2186	tl	14.99	0.23	0.08	6341204	tl	0.18	0.11	0.00	12618	tl	0.18	0.29	0.11
<i>gcut11d</i>	512	tl	11.36	0.83	0.52	9230	11.3	0.06	0.01	0.00	3425	tl	0.06	0.12	0.06
<i>gcut12d</i>	300	tl	13.31	0.60	0.35	11145	24.4	0.04	0.01	0.00	3647	tl	0.04	0.06	0.02
<i>HH</i>	1195	1.6	39.43	0.00	0.00	21499	6.8	40.07	0.00	0.00	0	0.3	0.00	0.00	0.00
<i>CW1</i>	134745	tl	12.45	8.27	0.60	338623	tl	8.90	7.49	0.00	247	tl	8.52	9.25	0.80
<i>CW2</i>	142631	tl	8.56	1.36	0.14	288872	tl	2.31	1.06	0.00	285	tl	1.56	8.24	6.78
<i>CW3</i>	62093	tl	8.06	4.11	0.09	103769	tl	4.15	3.58	0.00	96	tl	3.78	9.18	5.61
<i>Hchl2</i>	47786	tl	11.96	9.04	0.00	10920	tl	10.49	8.84	0.00	2	tl	1.20	3.03	1.85
<i>Hchl9</i>	60644	tl	12.91	9.49	0.79	54936	tl	9.86	8.62	0.00	56	tl	8.80	10.48	1.84
<i>2s</i>	2484	9.4	19.12	0.00	0.00	45331	76.5	16.63	0.00	0.00	60	35.2	1.54	1.54	0.00
<i>3s</i>	0	0.1	29.48	0.00	0.00	7	0.1	1.64	0.00	0.00	2	0.5	1.64	1.64	0.00
<i>A1s</i>	32717	140.4	10.95	0.00	0.00	3495	4.0	1.79	0.00	0.00	2902	tl	1.32	6.27	5.02
<i>A2s</i>	607952	tl	17.07	4.64	0.17	1629607	tl	5.65	3.68	0.00	3323	tl	5.34	5.90	0.60
<i>STS2S</i>	62731	tl	7.07	3.83	1.23	165381	tl	2.62	1.84	0.00	136	tl	0.87	8.44	7.63
<i>STS4S</i>	115150	tl	10.19	6.94	0.61	170209	tl	7.91	5.96	0.00	87	tl	1.23	4.15	2.96
<i>OF1</i>	208649	219.8	25.23	0.00	0.00	3656880	1249.8	21.42	0.00	0.00	11962	tl	1.03	1.03	0.00
<i>OF2</i>	897415	1170.2	19.21	0.00	0.00	383934	206.0	12.35	0.00	0.00	11107	tl	0.78	0.78	0.00
<i>W</i>	916284	tl	17.42	3.82	0.21	1961347	tl	4.21	2.63	0.00	10811	tl	4.10	4.34	0.26
<i>CHL1S</i>	78333	tl	14.93	10.63	0.00	11486	tl	13.32	11.27	0.72	43	tl	1.26	1.98	0.72
<i>CHL2S</i>	32108	86.3	25.33	0.00	0.00	961331	516.1	23.34	0.00	0.00	33	14.1	0.00	0.00	0.00
<i>A3</i>	220775	tl	8.10	0.63	0.00	14360	35.6	2.54	0.00	0.00	1491	tl	0.88	2.04	1.17
<i>A4</i>	272114	tl	21.17	12.75	0.00	131959	tl	18.17	13.10	0.00	13252	tl	0.62	1.23	0.62
<i>A5</i>	153745	tl	8.71	16.95	16.93	1161	29.3	4.54	0.00	0.00	226	tl	0.29	18.23	17.99
<i>CHL5</i>	927	1.6	14.17	0.00	0.00	2295	1.5	13.34	0.00	0.00	46	3.7	1.03	1.03	0.00
<i>CHL6</i>	65861	tl	17.25	14.47	0.00	14848	tl	16.06	14.89	0.52	35	tl	14.41	15.44	1.20
<i>CHL7</i>	40567	tl	12.11	9.97	0.00	9206	tl	11.55	10.91	1.19	2	tl	0.69	15.17	14.58
<i>CU1</i>	63105	tl	6.92	6.15	5.55	435883	tl	0.96	0.22	0.00	155	tl	0.66	6.52	5.89
<i>CU2</i>	170422	tl	10.62	5.05	0.25	298663	tl	6.84	4.61	0.00	748	tl	5.68	6.55	0.92
<i>Hchl3s</i>	95858	tl	10.89	4.05	0.00	431076	tl	8.45	3.32	0.00	359	tl	1.05	1.05	0.00
<i>Hchl4s</i>	1284	5.8	11.89	0.00	0.00	533973	tl	11.67	1.89	0.00	23	77.4	1.56	1.56	0.00
<i>Hchl6s</i>	87682	tl	13.67	10.56	0.00	21467	tl	11.69	10.15	0.29	36	tl	0.60	17.09	16.58
<i>Hchl7s</i>	28564	tl	18.83	15.87	0.00	500	tl	17.43	18.25	1.94	0	tl	13.54	13.88	0.39
<i>Hchl8s</i>	3030	6.9	44.85	0.00	0.00	855929	1006.9	45.86	0.00	0.00	0	1.6	0.00	0.00	0.00

Table 2: *Multi* case results

<i>instance</i>	Compact formulation		Pseudo-polynomial formulation		Exponential formulation		
	<i>vars</i>	<i>cons</i>	<i>vars</i>	<i>cons</i>	<i>cols</i>	<i>cols_N</i>	$ B = 2$
<i>gcut1d</i>	686637	8035	400	147	135	92	37.35
<i>gcut2d</i>	1480713	11801	2834	699	169	76	0.00
<i>gcut3d</i>	3403935	17895	5327	821	221	71	0.00
<i>gcut4d</i>	11591328	33059	12317	1281	432	130	35.60
<i>gcut5d</i>	637500	7747	483	115	150	101	29.81
<i>gcut6d</i>	1734927	12785	1245	264	178	98	30.05
<i>gcut7d</i>	4044597	19539	2932	603	320	171	46.41
<i>gcut8d</i>	8583078	28403	11723	1202	365	95	34.84
<i>gcut9d</i>	534282	7087	512	149	161	124	33.21
<i>gcut10d</i>	1064358	9977	1085	178	313	225	39.81
<i>gcut11d</i>	2590407	15603	4751	600	395	236	27.17
<i>gcut12d</i>	6681090	25019	11535	1416	354	88	42.38
<i>HH</i>	726	218	749	268	31	0	0.00
<i>CW1</i>	9282	826	6077	744	1362	1185	36.29
<i>CW2</i>	9480	788	7697	829	1928	1676	36.36
<i>CW3</i>	19206	1189	13603	1394	1315	1028	46.39
<i>Hchl2</i>	12798	932	29666	1880	445	22	0.00
<i>Hchl9</i>	13320	944	13565	1020	1064	754	42.11
<i>2s</i>	1260	283	1475	238	587	514	1.64
<i>3s</i>	8058	761	1591	222	90	1	0.00
<i>A1s</i>	8058	761	1609	289	1255	1152	38.89
<i>A2s</i>	5994	653	2154	378	937	850	41.49
<i>STS2S</i>	13326	963	7757	692	1386	1126	43.80
<i>STS4S</i>	5574	617	6812	801	1187	1013	40.91
<i>OF1</i>	1326	283	997	250	1111	1062	24.23
<i>OF2</i>	1404	295	1040	254	1645	1576	29.24
<i>W</i>	8163	761	2145	434	488	383	46.22
<i>CHL1S</i>	8970	783	25098	2023	1123	830	31.82
<i>CHL2S</i>	918	235	1146	253	240	172	23.53
<i>A3</i>	4839	569	3870	528	1952	1802	29.42
<i>A4</i>	3090	437	7438	1026	1616	1463	0.63
<i>A5</i>	4557	557	9858	1197	1749	1586	40.97
<i>CHL5</i>	867	223	582	133	199	142	6.38
<i>CHL6</i>	9435	807	23251	1782	1011	721	30.56
<i>CHL7</i>	12813	932	27909	1779	398	9	0.00
<i>CU1</i>	16266	1112	8121	783	1223	1013	40.38
<i>CU2</i>	13191	1006	6312	792	1338	1201	42.46
<i>Hchl3s</i>	4935	619	3818	736	1837	1756	13.06
<i>Hchl4s</i>	2184	391	3818	736	363	289	0.00
<i>Hchl6s</i>	7662	739	18672	1990	837	632	24.32
<i>Hchl7s</i>	18444	1117	83005	4970	476	22	0.00
<i>Hchl8s</i>	867	223	1802	383	73	0	0.00

Table 3: Formulation Features

<i>instance</i>	Compact formulation					Pseudo-polynomial formulation				
	<i>nodes</i>	<i>time</i>	LP_{gap}	OPT_{gap}	SOL_{gap}	<i>nodes</i>	<i>time</i>	LP_{gap}	OPT_{gap}	SOL_{gap}
<i>gcut1d</i>	238364	tl	36.69	0.44	0.00	0	0.0	0.25	0.00	0.00
<i>gcut2d</i>	299	tl	34.59	0.43	0.00	0	0.0	0.22	0.00	0.00
<i>gcut3d</i>	19	tl	14.06	1.26	1.16	0	0.0	0.05	0.00	0.00
<i>gcut4d</i>	10	424.1	19.25	0.00	0.00	0	0.1	0.06	0.00	0.00
<i>gcut5d</i>	0	25.3	30.81	0.00	0.00	0	0.0	0.16	0.00	0.00
<i>gcut6d</i>	471	tl	30.83	0.40	0.00	0	0.0	0.09	0.00	0.00
<i>gcut7d</i>	0	12.6	28.96	0.00	0.00	0	0.0	0.08	0.00	0.00
<i>gcut8d</i>	300	tl	19.97	0.26	0.00	0	0.1	0.10	0.00	0.00
<i>gcut9d</i>	481	54.7	16.46	0.00	0.00	0	0.0	0.56	0.00	0.00
<i>gcut10d</i>	1278	455.6	29.58	0.00	0.00	0	0.0	0.05	0.00	0.00
<i>gcut11d</i>	503	tl	22.96	0.29	0.00	0	0.0	0.25	0.00	0.00
<i>gcut12d</i>	276	tl	21.69	0.50	0.30	0	0.1	0.06	0.00	0.00
<i>HH</i>	0	0.0	41.65	0.00	0.00	0	0.0	42.39	0.00	0.00
<i>CW1</i>	480	1.9	15.32	0.00	0.00	0	0.1	8.51	0.00	0.00
<i>CW2</i>	230	0.9	10.27	0.00	0.00	121	0.3	2.02	0.00	0.00
<i>CW3</i>	120	1.8	10.83	0.00	0.00	0	0.3	4.09	0.00	0.00
<i>Hchl2</i>	180	3.1	13.68	0.00	0.00	0	2.2	11.63	0.00	0.00
<i>Hchl9</i>	10710	88.5	9.02	0.00	0.00	166	1.9	4.16	0.00	0.00
<i>2s</i>	0	0.0	18.60	0.00	0.00	0	0.0	12.32	0.00	0.00
<i>3s</i>	0	0.0	30.82	0.00	0.00	0	0.0	2.17	0.00	0.00
<i>A1s</i>	0	0.0	35.43	0.00	0.00	0	0.0	2.17	0.00	0.00
<i>A2s</i>	0	0.1	19.63	0.00	0.00	0	0.0	3.42	0.00	0.00
<i>STS2S</i>	562	10.4	9.93	0.00	0.00	50	0.2	2.94	0.00	0.00
<i>STS4S</i>	1709	7.2	12.90	0.00	0.00	30	0.4	8.03	0.00	0.00
<i>OF1</i>	0	0.0	27.30	0.00	0.00	0	0.0	23.35	0.00	0.00
<i>OF2</i>	1099	0.4	35.70	0.00	0.00	132	0.0	22.67	0.00	0.00
<i>W</i>	0	0.0	33.56	0.00	0.00	0	0.0	2.53	0.00	0.00
<i>CHL1S</i>	3487	14.3	16.35	0.00	0.00	461	21.7	12.29	0.00	0.00
<i>CHL2S</i>	0	0.0	28.53	0.00	0.00	0	0.0	26.70	0.00	0.00
<i>A3</i>	0	0.1	21.93	0.00	0.00	0	0.1	11.20	0.00	0.00
<i>A4</i>	0	0.2	23.56	0.00	0.00	0	0.2	17.97	0.00	0.00
<i>A5</i>	0	0.3	26.12	0.00	0.00	0	0.4	20.22	0.00	0.00
<i>CHL5</i>	0	0.0	36.23	0.00	0.00	0	0.0	17.50	0.00	0.00
<i>CHL6</i>	60	1.1	20.23	0.00	0.00	10	2.0	16.77	0.00	0.00
<i>CHL7</i>	521	3.3	14.93	0.00	0.00	260	8.3	13.09	0.00	0.00
<i>CU1</i>	10	0.6	14.03	0.00	0.00	0	0.1	6.36	0.00	0.00
<i>CU2</i>	160	1.3	13.10	0.00	0.00	0	0.2	6.37	0.00	0.00
<i>Hchl3s</i>	223	0.5	12.17	0.00	0.00	50	0.2	10.25	0.00	0.00
<i>Hchl4s</i>	24	0.1	13.35	0.00	0.00	58	0.2	13.80	0.00	0.00
<i>Hchl6s</i>	240	1.3	16.42	0.00	0.00	240	3.8	12.73	0.00	0.00
<i>Hchl7s</i>	28429	322.8	9.57	0.00	0.00	1530	tl	19.43	17.81	12.50
<i>Hchl8s</i>	0	0.0	38.20	0.00	0.00	0	0.0	46.50	0.00	0.00

Table 4: *Mono* case results

4 Conclusions

In this paper, we consider the Two-Dimensional Cutting Stock Problem where stock of different sizes is available, and a set of rectangular items has to be obtained through two-stage guillotine cuts. We present three different formulations for the case of different stock sizes. The first one is an extension of the Bin Packing model by Lodi et al. [Lodi et al. \(2004\)](#). The second one is proposed by Silva et al. [Silva et al. \(2010\)](#), and we study its performance in the case of multiple stock size. Finally, the third one is the adaptation of the exponential-size formulation proposed by Gilmore and Gomory for cutting stock problems to the case of multiple stock size. This last formulation needs branch-and-price techniques to be solved while the previous two can be solved directly by using a generic MIP solver. In the context of branch-and-price, we test a branching rule specifically conceived for the 2DCS. The paper is concluded by an extensive computational analysis that evaluates the performances of the presented mathematical formulations. It highlights the quality of each formulation in terms of number of solved instances, computing time, integrality gap, quality of the continuous relaxation and quality of the best feasible solutions for instances not solved to optimality.

5 Acknowledgments

The authors would like to thank Andrea Lodi, Michele Monaci and José M.V. Valério de Carvalho for stimulating discussions on 2D cutting; Charlotte Mitchell and Moushumi Bhadra for their linguistic assistance; and two anonymous referees for their careful reading and helpful comments.

References

- R. Alvarez-Valdes, A. Parajon, and J.M. Tamarit. A computational study of lp-based heuristic algorithms for two-dimensional guillotine cutting stock problems. *OR Spektrum*, 24:179–192, 2002.
- G.F. Cintra, F.K. Miyazawa, Y. Wakabayashi, and E.C. Xavier. Algorithms for two-dimensional cutting stock and strip packing problems using dynamic programming and column generation. *European Journal of Operational Research*, 191:61–85, 2008.
- J.M. Valério de Carvalho. Lp models for bin packing and cutting stock problems. *European Journal of Operational Research*, 141:253 – 273, 2002.
- H. Dyckhoff. A new linear programming approach to the cutting stock problem. *Operations Research*, 29: 1092–1104, 1981.
- F. Furini, E. Malaguti, R. Medina Durán, A. Persiani, and P. Toth. A column generation heuristic for the two-dimensional two-staged guillotine cutting stock problem with multiple stock size. *European Journal of Operational Research*, 218:251–260, 2012.
- P.C. Gilmore and R.E. Gomory. Multistage cutting stock problems of two and more dimensions. *Operations Research*, 13:94–120, 1965.
- M. Hifi and C. Roucairol. Approximate and exact algorithms for constrained (un) weighted two-dimensional two-staged cutting stock problems. *Journal of Combinatorial Optimization*, 5:465–494, 2001.

- A. Lodi and M. Monaci. Integer linear programming models for 2-staged two-dimensional knapsack problems. *Mathematical Programming*, 94:257–178, 2003.
- A. Lodi, S. Martello, and M. Monaci. Two-dimensional packing problems: a survey. *European Journal of Operational Research*, 141:241–252, 2002.
- A. Lodi, S. Martello, and D. Vigo. Models and bounds for two-dimensional level packing problems. *Journal of Combinatorial Optimization*, 8:363–379, 2004.
- R. Macedo, C. Alves, and J.M. Valério de Carvalho. Arc-flow model for the two-dimensional guillotine cutting stock problem. *Computers & Operations Research*, 37:991 – 1001, 2010.
- E. Malaguti, R. Medina Durán, and P. Toth. Approaches to real world two-dimensional cutting problems. Technical Report OR/12/8, DEIS - University of Bologna, 2012.
- S. Martello and P. Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, Chichester, 1990.
- OR-Library. <http://people.brunel.ac.uk/~mastjjb/jeb/info.html>.
- D. Pisinger and M. Sigurd. The two-dimensional bin packing problem with variable bin sizes and costs. *Discrete Optimization*, 2:154–167, 2005.
- D. Pisinger and M. Sigurd. Using decomposition techniques and constraint programming for solving the two-dimensional bin-packing problem. *INFORMS Journal on Computing*, 19:36–51, 2007.
- J. Puchinger and G. R. Raidl. Models and algorithms for three-stage two-dimensional bin packing. *European Journal of Operational Research*, 183:1304–1327, 2007.
- J. Riehme, G. Scheithauer, and J. Terno. The solution of two-stage guillotine cutting stock problems having extremely varying order demands. *European Journal of Operational Research*, 91:543–552, 1996.
- E. Silva, F. Alvelos, and J.M. Valério de Carvalho. An integer programming model for two- and three-stage two-dimensional cutting stock problems. *European Journal of Operational Research*, 205:699 – 708, 2010.
- IBM ILOG CPLEX v12.4. User’s Manual for CPLEX, <http://www-01.ibm.com/software/integration/optimization/cplex-optimizer/> (2012).
- F. Vanderbeck. Computational study of a column generation algorithm for bin packing and cutting stock problems. *Mathematical Programming*, 86:565–594, 1999.
- F. Vanderbeck. On Dantzig-Wolfe decomposition in integer programming and ways to perform branching in a branch-and-price algorithm. *Operations Research*, 48:111–128, 2000.
- F. Vanderbeck. A nested decomposition approach to a three-stage, two-dimensional cutting-stock problem. *Management Science*, 47:864–879, 2001.
- G. Wäscher, H. Haussner, and H. Schumann. An improved typology of cutting and packing problems. *European Journal of Operational Research*, 183:1109–1130, 2007.